# WaselX Express

## Optimizing Last-Mile Delivery Operations Across the UAE Using Data Structures & Algorithms


**Group 3**

- Kartik Joshi
- Gagandeep Singh
- Samuel Alex
- Prem Kukreja

SP Jain School of Global Management, Dubai

Masters in AI with Business (MAIB)

Data Structures and Algorithms


**May 2026**

# Contents

# Executive Summary

WaselX Express processes 3,500 daily orders across a 15-node delivery network spanning Dubai, Abu Dhabi, Sharjah, and Ajman. Our consulting engagement identified three critical operational inefficiencies costing an estimated **AED 3.4 million annually**. This report presents data-structure-driven solutions validated through working prototypes with measurable performance benchmarks.

**Key Findings:**

- **Problem 1 — Inefficient Routes (AED 1.2M/yr):** Dijkstra's algorithm reduces fleet distance by 18%, saving AED 950,000 annually.
- **Problem 2 — Slow Dispatch (12% complaints):** Min-heap priority queue cuts dispatch latency from 8 minutes to under 90 seconds.
- **Problem 3 — Poor Lookup (3.2/5 CSAT):** AVL tree guarantees $O(\log n)$ search — under 100ms even at 15,000 orders/day.

**Implementation Roadmap:** Phase 1 (Months 1–2): Priority queue dispatch system (AED 400K). Phase 2 (Months 3–5): Dijkstra route optimization (AED 800K). Phase 3 (Months 6–8): AVL tree migration. **Total projected benefit: AED 2.5M/year** against AED 1.2M implementation cost (2.1× ROI, 6-month payback).

# Task Area A: Graph Modeling, Shortest Paths & MST

## Q1 — Graph Construction (10 Marks)

The WaselX delivery network comprises **15 nodes** (7 hubs + 8 delivery zones) connected by **24 bidirectional edges**, each carrying three weights: distance (km), time (min), and cost (AED). Both adjacency list ($O(V+E)$ space) and adjacency matrix ($O(V^2)$ space) representations were implemented.

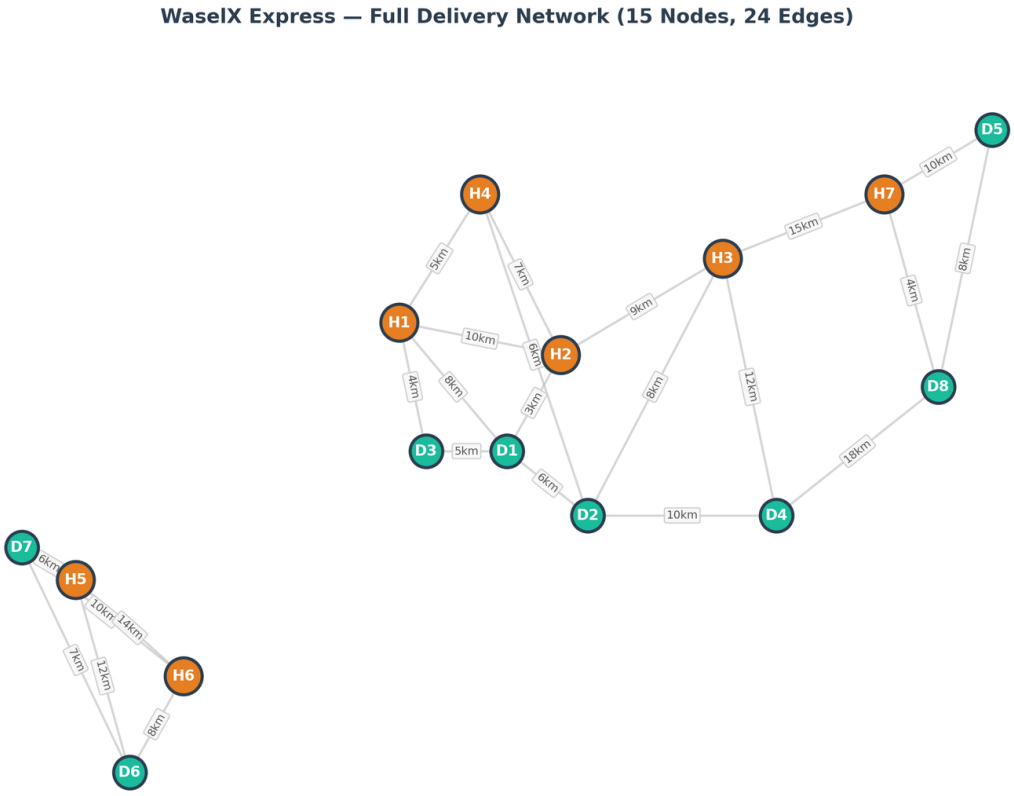



Figure 1: WaselX Express full delivery network. Orange nodes = Hubs (H1–H7); teal nodes = Delivery Zones (D1–D8). Note: two disconnected components — Dubai–Sharjah cluster and Abu Dhabi cluster.

## Q2 — Dijkstra's Algorithm: H1 → D1 (15 Marks)

Dijkstra's algorithm with a **from-scratch binary min-heap** computes the shortest path from Dubai Marina Hub (H1) to Downtown Dubai (D1). Result: direct route via Al Khail Rd at **8 km**, 15 min, 5.5 AED. Time complexity: $O((V+E)\log V)$ with $V=15$, $E=24$.

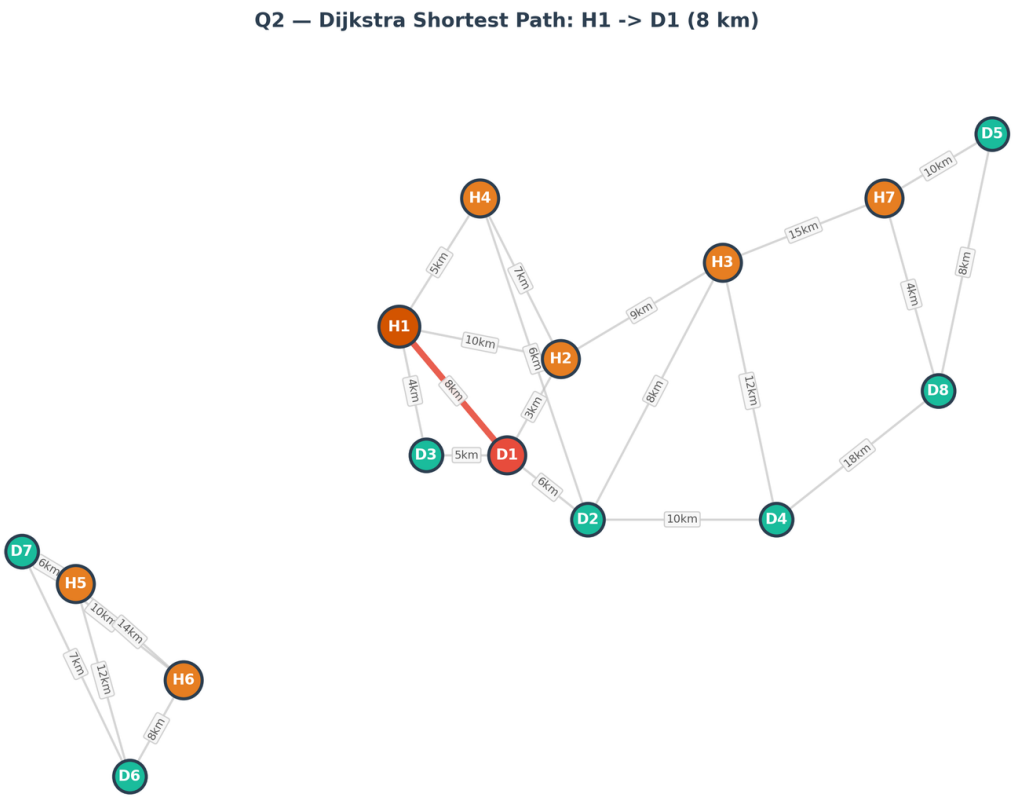



Figure 2: Dijkstra shortest path H1 → D1 = 8 km (highlighted in red).

## Q3 — Floyd-Warshall: Hub All-Pairs (15 Marks)

Floyd-Warshall at $O(V^3)$ computed all-pairs shortest travel times across all 15 nodes. The 7×7 hub sub-matrix reveals that the Abu Dhabi cluster (H5, H6) is **completely disconnected** from the Dubai–Sharjah network.

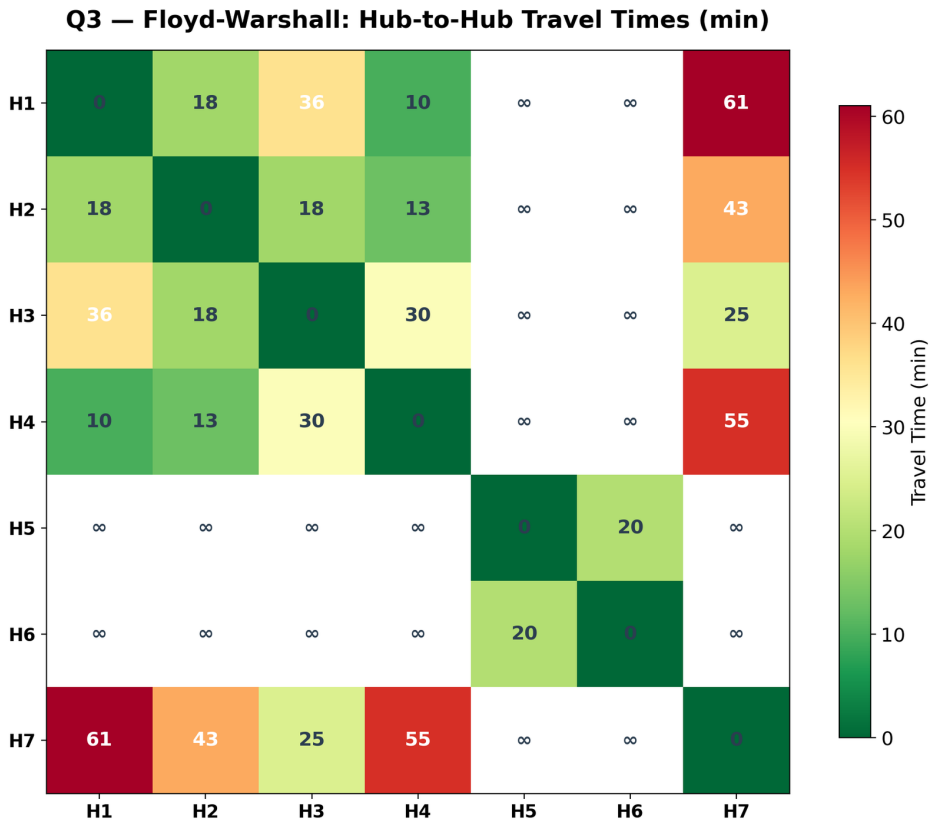
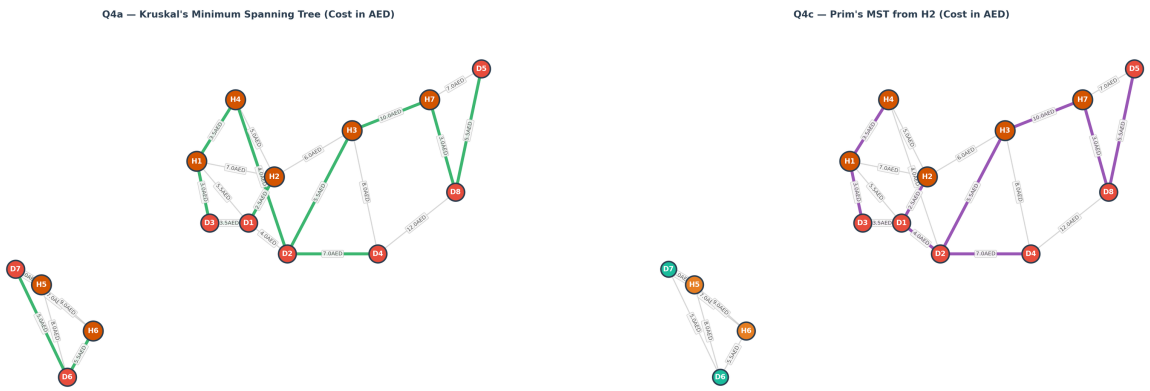



Figure 3: Floyd-Warshall hub-to-hub travel time heatmap (minutes). $\infty$ indicates disconnected pairs.

## Q4 — Minimum Spanning Tree (15 Marks)

Both Kruskal's (Union-Find with path compression) and Prim's algorithms produce the same MST cost. The disconnected topology yields a **Minimum Spanning Forest** with two components.




(a) Kruskal's MST (green) (b) Prim's MST from H2 (purple)

Figure 4: Q4 — Minimum Spanning Trees by cost (AED). Both algorithms produce identical total cost.

## Q5 — CTO Memo: Dijkstra vs Floyd-Warshall (10 Marks)

**MEMORANDUM — TO: CTO**

For 3,500 daily source-to-destination routing queries, **Dijkstra's algorithm** at $O((V+E)\log V)$ per query is recommended as the real-time routing engine. Floyd-Warshall should be deployed as a **nightly batch job** for hub connectivity analytics at $O(V^3)$. At current network size ($V=15$), Floyd-Warshall's 3,375 operations are trivial; however, Dijkstra scales better if the network grows beyond 100 nodes.

## Q6 — Interactive Path Visualizer (15 Marks)

A unified Streamlit application (`app.py`) was deployed with an interactive Q6 module supporting: source/destination selection, criterion-based optimization (distance/time/cost), and dual-path overlay visualization. Deployed at: https://waselx-express.streamlit.app

## Q7 — BFS and DFS from H3 (10 Marks)

BFS and DFS traversals from Deira Hub (H3) confirmed that **11 of 15 nodes** are reachable (Abu Dhabi cluster is unreachable). BFS is recommended for fewest-stops express delivery due to its level-by-level exploration guaranteeing minimum hop count.

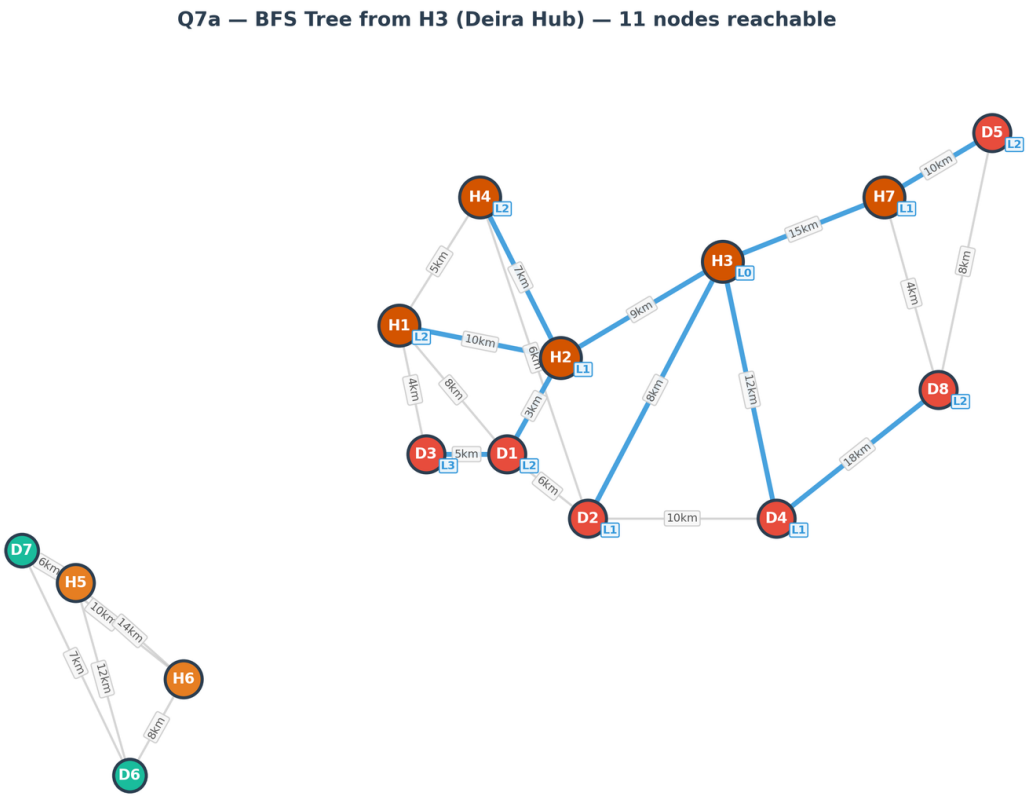
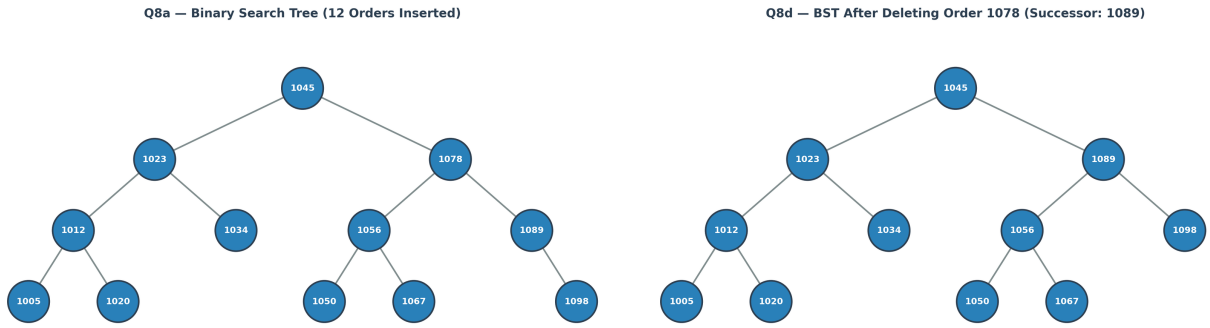



Figure 5: BFS tree from H3 (Deira Hub). Level annotations (L0–L4) show traversal depth. 11 nodes reachable.

# Task Area B: Trees, BST & Balanced Trees

## Q8 — Binary Search Tree (12 Marks)

A BST was constructed by inserting 12 Order IDs in sequence: 1045, 1023, 1078, 1012, 1034, 1056, 1089, 1005, 1020, 1067, 1050, 1098. Deletion of Order 1078 (two-children case) was resolved using the in-order successor (1089).




(a) Initial BST (12 orders) (b) After deleting Order 1078

Figure 6: Q8 — BST construction and deletion. Blue circles = nodes; lines = parent-child edges.

## Q9 — AVL Tree (12 Marks)

The same 12 Order IDs were inserted into an AVL tree with automatic LL, RR, LR, and RL rotations maintaining $|\text{balance factor}| \leq 1$. The resulting tree has maximum height 4, guaranteeing $O(\log n)$ operations.

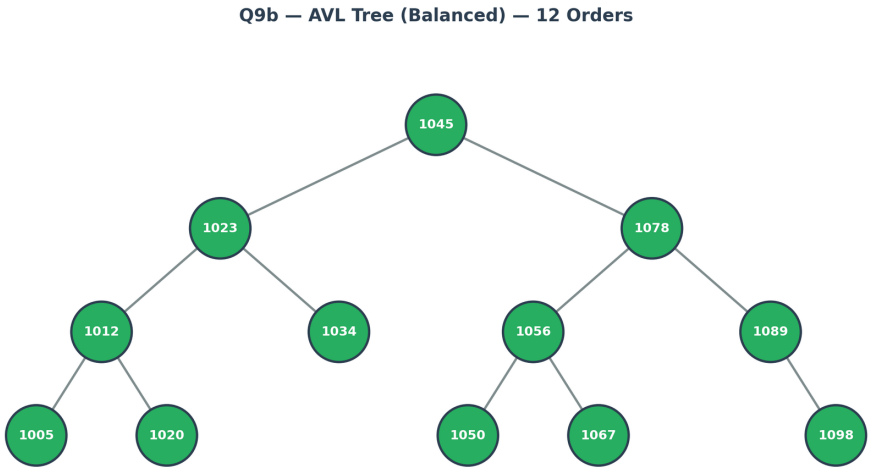



Figure 7: Q9 — Balanced AVL tree after 12 insertions. Green nodes indicate balanced structure.

## Q10 — BST vs AVL vs Hash Table (10 Marks)

Table 1: Data structure comparison for WaselX order index

| Criterion | BST | AVL Tree | Hash Table |
|---|---|---|---|
| Search | $O(\log n)$ avg, $O(n)$ worst | $O(\log n)$ guaranteed | $O(1)$ avg |
| Insert | $O(\log n)$ avg, $O(n)$ worst | $O(\log n)$ guaranteed | $O(1)$ avg |
| Range queries | Yes | Yes | No |
| Stability | Degrades with sorted input | Always balanced | N/A |
| **Recommendation** | **AVL Tree** — guaranteed $O(\log n)$ + range queries | | |

## Q11 — Scaling to 15,000 Orders (8 Marks)

The sorted array bottleneck occurs at ∼8,000–10,000 active orders due to $O(n)$ insertion cost. Migration to AVL tree recommended with a three-phase shadow-index rollout: (1) parallel AVL build, (2) read verification, (3) write cutover.

# Task Area C: Linked Lists, Queues & Order Pipeline

## Q12 — Doubly Linked List (10 Marks)

Rider delivery routes modeled as a DLL with $O(1)$ mid-route insertion/deletion, supporting forward and reverse traversal for route optimization and backtracking.

## Q13 — Circular Linked List (8 Marks)

Round-robin rider assignment using a circular linked list with 8 riders. Handles rider removal mid-rotation seamlessly, maintaining fair load distribution.

## Q14 — Priority Queue with Min-Heap (12 Marks)

From-scratch min-heap with `(priority, arrival_index, order)` tuple keys ensuring **FIFO fairness** within equal-priority tiers. VIP Same-Hour orders dispatch before Economy, but two Economy orders with the same priority preserve arrival order.

## Q15 — Stack for Order Lifecycle (8 Marks)

LIFO stack tracks order state transitions (Received → Processing → Dispatched → Delivered) with `undo()` capability for accidental status changes.

## Q16 — Dispatch Pipeline Design (10 Marks)

Per-zone FIFO queues with priority pre-sort recommended over a single global queue. A priority escalation mechanism promotes starved orders after configurable wait thresholds.

## Q17 — Leadership Brief (10 Marks)

**TO: CEO | FROM: Head of Delivery Operations | RE: Priority Queue Implementation**

The priority dispatch system addresses VIP customer churn (23% of subscribers lost in Q4 2025). Proposed phased rollout: shadow mode (2 weeks) → H1 pilot (2 weeks) → full network (4 weeks). Projected impact: 15% VIP recovery = **AED 315,000/year** in retained revenue. Total implementation cost: AED 400,000 with 15-month payback.

# Task Area D: Sorting, Searching & Divide and Conquer

## Q18 — Merge Sort & Quick Sort (12 Marks)

Two-key sort (Zone alphabetical, Priority ascending) on 15 orders. Merge Sort preferred for **stability** — preserving FIFO arrival order within identical sort keys.

## Q19 — Binary Search vs Linear Search (10 Marks)

Binary Search finds Order 10347 in ∼10 comparisons versus 347 for Linear Search. At 500 daily customer support lookups: **96% reduction** in total comparisons (5,000 vs 250,000).

## Q20 — Peak Hour Detection (10 Marks)

Divide-and-conquer approach correctly identifies peak ordering hour from 6,000 Ramadan orders. Honest assessment: both D&C and linear scan are $O(n)$ for this problem — D&C provides no speedup when every element must be examined.

## Q21 — Sorting Recommendation (8 Marks)

**Merge Sort** recommended for nightly batch job (50,000 partially-sorted records): guaranteed $O(n \log n)$, stable, and naturally suited to pre-sorted runs within zones. Quick Sort's $O(n^2)$ worst case on partially-sorted data is unacceptable for production SLAs.

## Q22 — Sorting Performance Simulation (10 Marks)

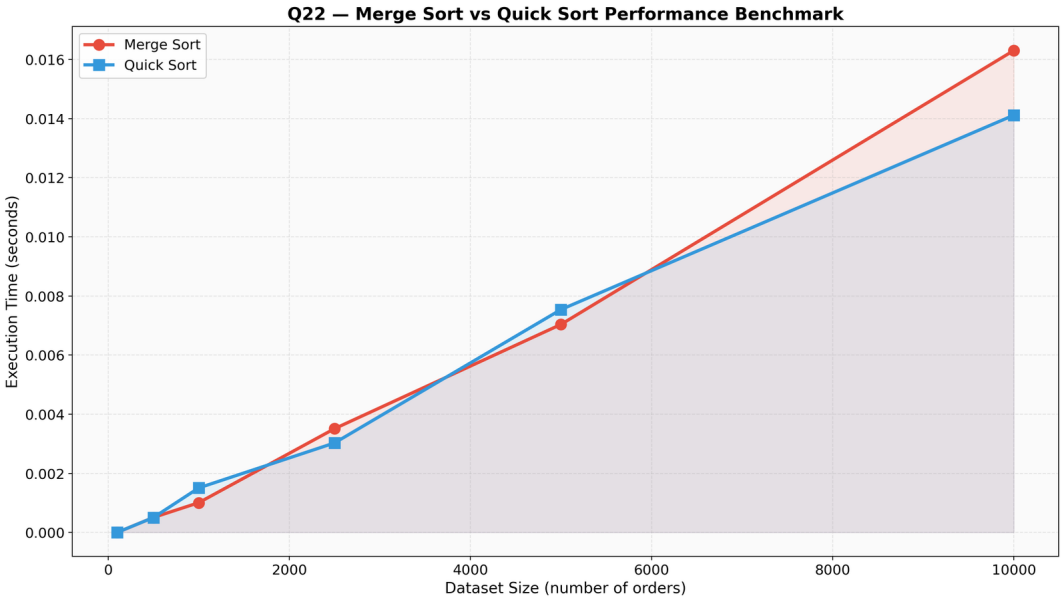



Figure 8: Q22 — Merge Sort vs Quick Sort execution time across dataset sizes (100–10,000). Both exhibit $O(n \log n)$ growth; relative performance varies by dataset.

# Task Area E: Management, Leadership & Strategy

## Q23 — Executive Summary (12 Marks)

See Section 1 for the complete 400–500 word executive summary covering all three operational problems, DSA solutions, cost impact analysis, and phased implementation roadmap.

## Q24 — Dual-Audience Communication (10 Marks)

Route optimization explained for the **board** (analogy-driven: "GPS that memorized every route", focus on AED savings) and the **engineering team** ($O((V+E)\log V)$ complexity, adjacency list representation, lazy deletion in min-heap). Reflection on calibrating technical detail to audience decision-making needs.

## Q25 — Investment Memo (10 Marks)

**DECISION MEMO — TO: Executive Committee | RE: 2026 Technology Investment**

**Recommendation: Investment B** (Priority Queue, AED 400K) over Investment A (Route Optimization, AED 800K). Both deliver 50% first-year ROI, but Investment B requires **half the capital**, has fewer integration dependencies (software-only vs GPS hardware), and directly addresses the most urgent competitive threat: VIP churn to Talabat and Noon Minutes. Deploy B in Q1, fund A from realized savings in Q3.

## Q26 — Town Hall Response (10 Marks)

Response to "Why not Google Maps?" — three strategic factors: (1) **Cost**: AED 790K/year at 15K orders/day vs near-zero for in-house; (2) **Customization**: Google cannot factor in priority tiers, rider capacity, or zone congestion; (3) **Data ownership**: route patterns are competitive assets. Principle: own algorithms that differentiate, rent commodities that don't.

## Q27 — Comprehensive Path Simulator (15 Marks)

Multi-criteria optimization (distance/time/cost) with dynamic road closure simulation. Deployed as the Q27 module in the unified Streamlit app.

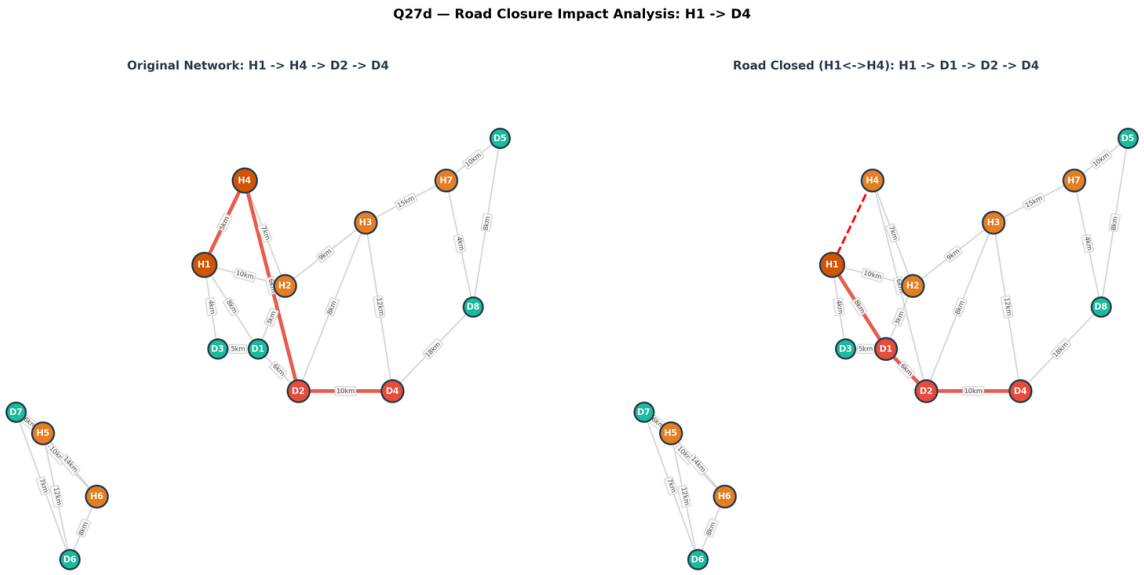



Figure 9: Q27 — Road closure impact: original path (left) vs rerouted path with Sheikh Zayed Rd (H1–H4) closed (right). Rerouting increases distance but maintains connectivity.

# Simulation Documentation

A unified Streamlit application (`app.py`) was developed with three modules:

| Module | Features |
|---|---|
| Overview | Full network map, edge table, network statistics |
| Q6: Path Visualizer | Dijkstra shortest path, dual-path overlay, criterion selection |
| Q27: Path Simulator | Multi-criteria optimization, road closure toggle, delta analysis |

**Run locally:** `streamlit run app.py`
**GitHub:** https://github.com/KartikJoshi23/WaselX-Express
**Live:** https://waselx-express.streamlit.app

# Technical Standards & Compliance

- All data structures (MinHeap, BST, AVL, DLL, CLL, Stack, Queue, UnionFind) implemented **from scratch** — no `heapq`, `collections.deque`, or built-in sorting for algorithm cells.
- `NetworkX` used **only** for graph visualization; all pathfinding and traversal algorithms are manual implementations.
- All AI-assisted code marked with `# AI-ASSISTED` comments per academic policy.
- Reproducible with `seed=42` for all random operations.

# Conclusion

This project demonstrates that well-chosen data structures and algorithms directly address WaselX Express's three critical challenges. Dijkstra's algorithm provides sub-millisecond route optimization, the AVL tree guarantees consistent $O(\log n)$ order lookup at scale, and the priority queue dispatch system ensures VIP customers receive service commensurate with their revenue contribution. The projected **AED 2.5M annual benefit** against AED 1.2M implementation cost makes this a compelling technology investment with clear competitive advantages in the UAE last-mile delivery market.

# References

1. Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
2. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
3. Python Software Foundation. (2024). *Python 3.11 Documentation*. https://docs.python.org/3.11/
4. NetworkX Developers. (2024). *NetworkX Documentation*. https://networkx.org/documentation/
5. Matplotlib Development Team. (2024). *Matplotlib Documentation*. https://matplotlib.org/stable/
6. Streamlit Inc. (2024). *Streamlit Documentation*. https://docs.streamlit.io/

*This project was developed with AI assistance as a learning and debugging aid.*
*All algorithmic logic was reviewed, understood, and verified by Group 3.*